Architecting Knowledge Assistants

Routing · Memory · GraphRAG — From Theory to Working Code

Brainster AIatWork · April 2026

Maio Labs

THE SPEAKER

Rui Machado.

Co-founder & Chief Technology Officer

Maio Labs — AI products & services lab focused on productivity

BOOK AUTHOR

Analytics Engineering with SQL and dbt (O'Reilly, 2023)
Advanced SQL: Modern Data Solutions (O'Reilly, 2026)
Syncfusion Succinctly ebooks (2014, 2015)

EXPERIENCE

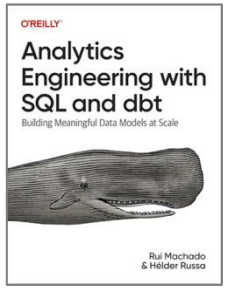
Nike · Facebook · H&M · Jumia · ShopAI

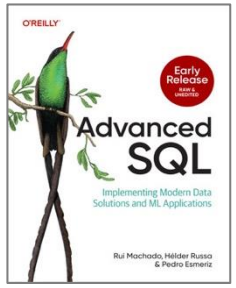
AI TEACHING & TRAINING

Executive and technical sessions for companies and institutions across Europe
AI Specialist Professor at ISLA (Instituto Superior de Gaia)

15+

YEARS EXPERIENCE





BOOKS PUBLISHED

50+

COMPANIES TRAINED

What we cover today.

TALK **01** The architecture gap — naive RAG vs knowledge system

TALK **02** Where GraphRAG fits — and what local vs global means

TALK **03** Two production cases — numbers, not slides

TALK **04** The Karpathy Pattern — the smallest thing that works

BUILD **05** Hands-on: Router + Memory + GraphRAG in one notebook

BUILD **06** Production path — from Colab to owned system

15 years of knowledge assistants

Each era got closer to answers. None of them got all the way.

2010

SEARCH ERA

Google + 10 blue links

You ask. You get a list. You read and synthesize yourself. The work still lives with the human.

2011

VOICE ERA

Siri, Alexa, Cortana

Intent classifiers over hand-built skills. Great for timers and weather. Hits a wall beyond.

2022

LLM CHAT

ChatGPT breaks through

Fluent answers, zero grounding. Confidently wrong. No access to your data. Stale by training cutoff.

2023

RAG

Retrieve, then generate

Bolt vector search onto the LLM. Ground answers in real docs. The default pattern for enterprise AI.

2024+

AGENTIC RAG

Routing, memory, graphs

RAG as a system: multiple retrievers, tools, memory, graphs. What we're here to talk about.

THE TRAJECTORY

Every era moved the synthesis work from the user to the machine. RAG is where the machine finally grounds itself.

Each era solved yesterday's problem and created tomorrow's.

What is RAG?

Retrieval-Augmented Generation — the four-step loop every RAG system runs.

// STEP 01

CHUNK

Split your documents into small, searchable pieces — usually a few hundred tokens each.

// STEP 02

EMBED

Turn each chunk into a vector with an embedding model, then store in a vector DB.

// STEP 03

RETRIEVE

Embed the user's question, find the top-k most similar chunks by vector distance.

// STEP 04

GENERATE

Stuff those chunks into the LLM's context along with the question. Answer.

THE PROMISE

Give the model access to your data without fine-tuning. Ground answers in sources. Update knowledge by updating the index — not the weights.

The difference is never the model.

NAIVE RAG

"What's our Q3 revenue?"

- ✗ Searches document vectors
- ✗ Retrieves stale 2023 report
- ✗ Confident but wrong answer
- ✗ No memory of prior context
- ✗ Same path for every question

KNOWLEDGE SYSTEM

"What's our Q3 revenue?"

- ✓ Detects: numerical / live data query
- ✓ Routes to BI tool via MCP
- ✓ Returns live data + citation
- ✓ Remembers prior context
- ✓ Right source for right question

Knowledge System ≠ RAG

RAG is a retrieval technique. A Knowledge System is the full stack around it.

// A TECHNIQUE

RAG

One retrieval pattern. Chunks in, answer out.

WHAT YOU BUILD

- Vector DB + embedder + LLM prompt
- One retriever, one corpus, one shot
- Stateless: no memory between turns
- Evals focus on retrieval accuracy
- Ships in a weekend

// A PRODUCT

Knowledge System

The system that answers questions over your org's knowledge.

WHAT YOU BUILD

- Routing across many retrievers & tools
- Memory, user context, permissions
- Graphs for entities & relationships
- Evals on answers, not just retrieval
- Ships as a product, not a demo

THE SHIFT **RAG is one ingredient. A Knowledge System is the meal, and the kitchen it's cooked in.**

Everything after this slide is about building the kitchen.

Classic RAG retrieves. It does not reason.

VECTOR RAG

chunk A "rates rose 2023"

chunk B "portfolio value..."

chunk C "mortgage defaults"

chunk D "Q3 revenue €42M"

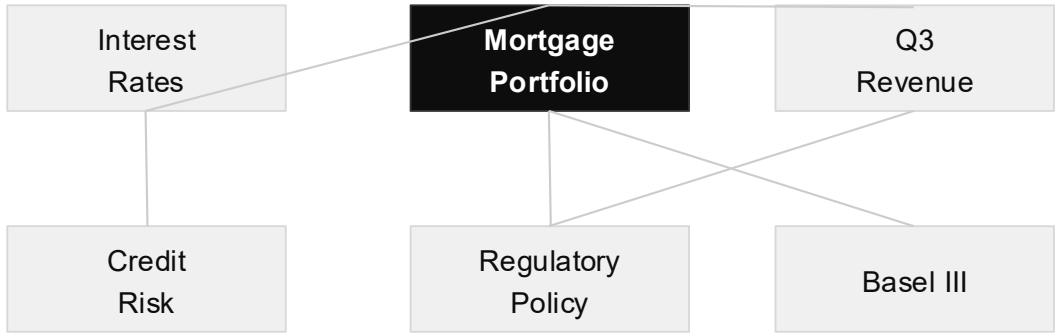
chunk E "credit risk policy..."

chunk F "Basel III ratios..."

Query → similarity → nearest chunk → answer ✗

Chunks are isolated. The answer spans all six.

KNOWLEDGE GRAPH



Query → traverse relationships → multi-hop answer ✓

All nodes connected. The answer emerges from the structure.

Every real assistant is some version of this.

1 ROUTER

Decides:
retrieve?
which source?
how deep?

→

2 RETRIEVAL SURFACES

RAG / Graph
SQL / APIs
MCP Tools

→

3 CONTEXT ASSEMBLY

Fuse, dedupe
rerank
compress

→

4 GENERATE +VALIDATE

Self-check
cite sources
escalate

→

5 MEMORY LAYER

Session / user
world memory
writes back

↑ GraphRAG lives here

Vector RAG or GraphRAG.

The numbers make it simple.

43–50%
multi-hop accuracy
vector RAG

80–91%
multi-hop accuracy
GraphRAG

10×
cheaper with
LightRAG

Microsoft Research, 2024 · HKUST, 2024

Use GraphRAG when...

Questions require linking across documents
Global synthesis — "what are the themes?"
Domain has explicit relations: contracts, tickets, legal
Explainability and auditability required

Stick with vector RAG when...

Questions are factual, single-document
Corpus changes frequently
Small corpus or tight latency budget
"Our users just need FAQs"

When the question spans 22 data sources.

The challenge

Alzheimer's research: genes, proteins, drugs, clinical trials, EHRs across 22 sources.

Solution — KRAGEN: knowledge graph + RAG + Graph-of-Thoughts. The LLM decomposes complex questions into sub-problems and traverses the graph.

22 data sources integrated

GoT Graph-of-Thoughts reasoning

OSS Open-source on GitHub

Matsumoto et al. — Cedars-Sinai / Bioinformatics · June 2024

Extract Knowledge from Knowledge Graph
Using Cypher Query Language

Convert to Natural Language Knowledge
Cypher Query results are turned into English sentences

Vectorize Natural Language
Using local or OpenAI models such as the text-embedding-ada-002 model

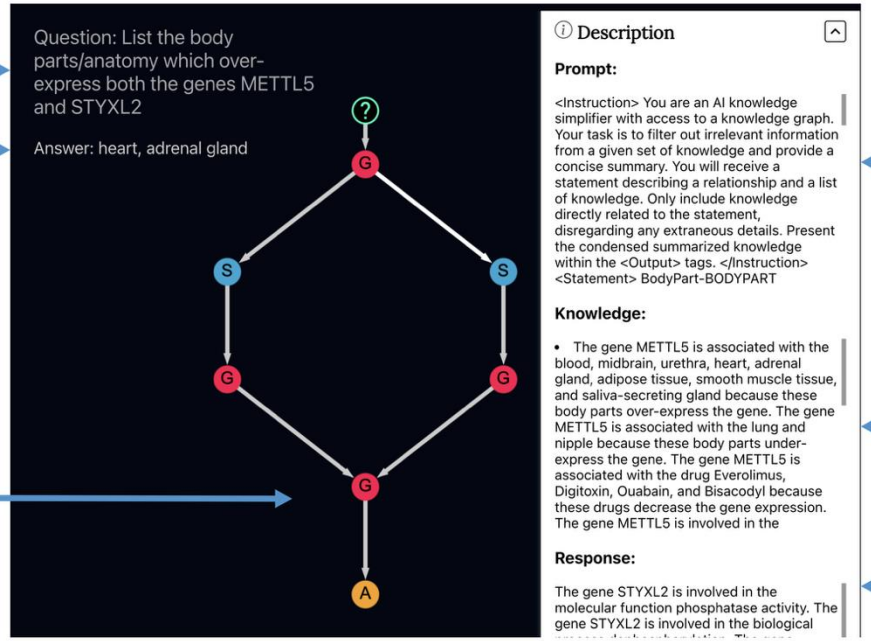
Upload to Vector DB
Use a REST API to upload the vectorized data onto a Weaviate Vector Database

GUI + RAG API
Python Graph of Thoughts and RAG Backend and a React Frontend

User Query

GoT Answer

Interactive GoT



GoT LLM Prompt

Knowledge From RAG

GoT LLM Response

Cedars Sinai

// PATTERNS / SPECTRUM

Folder of markdown ↔ Graph of entities

Two ends of the same spectrum. Pick based on where your data lives — not on hype.

// MONDAY MORNING

Karpathy pattern

Folder of .md files + LLM as librarian.

STRUCTURE

— Files + folders. No index, no embeddings
— Retrieval = LLM reads the tree
— You edit knowledge in a text editor
— Breaks past ~context window of content
— Great up to thousands of docs, one author

// ENTERPRISE SCALE

GraphRAG approach

Knowledge graph + vector retrieval + traversal.

STRUCTURE

— Entities + relationships extracted by LLM
— Retrieval = traverse + vector search hybrid
— Reindex on update. Pipeline to maintain
— Scales to millions of docs, many authors
— Wins when answers need to connect dots

WHEN TO PICK WHICH

Karpathy if one human can still read the docs. GraphRAG if relationships between entities are the answer.

02

Build it.

LightRAG. GraphRAG without the complexity.

01

One line to install

pip install lightrag-hku

No Neo4j, no Docker. Runs free in Colab.

02

10× fewer tokens

Two-level retrieval at dramatically lower cost than Microsoft GraphRAG. HKUST, Oct 2024.

03

Multiple graph backends

NetworkX (dev) → FalkorDB or Neo4j (prod). One config line to migrate.

04

Local + global out-of-the-box

Same API handles point questions and thematic synthesis. No extra config.

One notebook. Three systems. Connected.

PART A

The Router

Box 1

— LLM classifies each question

— 4 routes: docs / SQL / web / graph

— Logs every decision

PART B

Memory Layer

Box 5

— Session — this conversation

— User — verbatim preferences

— World — writes back to graph

PART C

GraphRAG

Box 2 — LightRAG

— Ingest → entities + relations

— Local: entity traversal

— Global: community synthesis

One assistant. Four routing paths.

"What's our return policy?"	→	`search_documents`	*Vector RAG — Chroma*
"What were yesterday's sales?"	→	`query_sales_data`	*SQL — DuckDB live data*
"What did Anthropic announce?"	→	`search_web`	*Web — Tavily*
"What links these 50 contracts?"	→	`graphrag_query`	*LightRAG global — new ✦*

```
router_chain = prompt | llm | StrOutputParser()   # returns one of the four route strings above
```

Three memory types.
One rule: write verbatim.

SESSION

Resets at conversation end

Last query, routing decisions, intermediate results. — Keep last 5 exchanges.

```
state["session_memory"].append({"role":"user","content":query})
```

USER

Persists across sessions

Preferences, role, language, past context. — Write verbatim — never interpret.

```
state["user_memory"][user_id] = {"role":"CFO","prefers":"bullets"}
```

WORLD

Shared — updates the graph

Facts all users should know — writes back to LightRAG. — Trigger on high-confidence facts.

```
await rag.ainsert(f"FACT: {new_fact}")  # writes to LightRAG
```

LightRAG in 4 steps. Runs in Colab.

01 Install + configure

```
pip install lightrag-hku openai
from lightrag import LightRAG, QueryParam
from lightrag.llm.openai import gpt_4o_mini_complete
```

3 lines.

02 Initialise + ingest

```
rag = LightRAG(working_dir="./storage",
               llm_model_func=gpt_4o_mini_complete)
await rag.ainsert(open("corpus.txt").read())
```

Extracts entities, builds graph, generates community summaries.

03 Local query

```
result = await rag.aquery(
  "Who signed the GlobalBank contract?",
  param=QueryParam(mode="local")
)
```

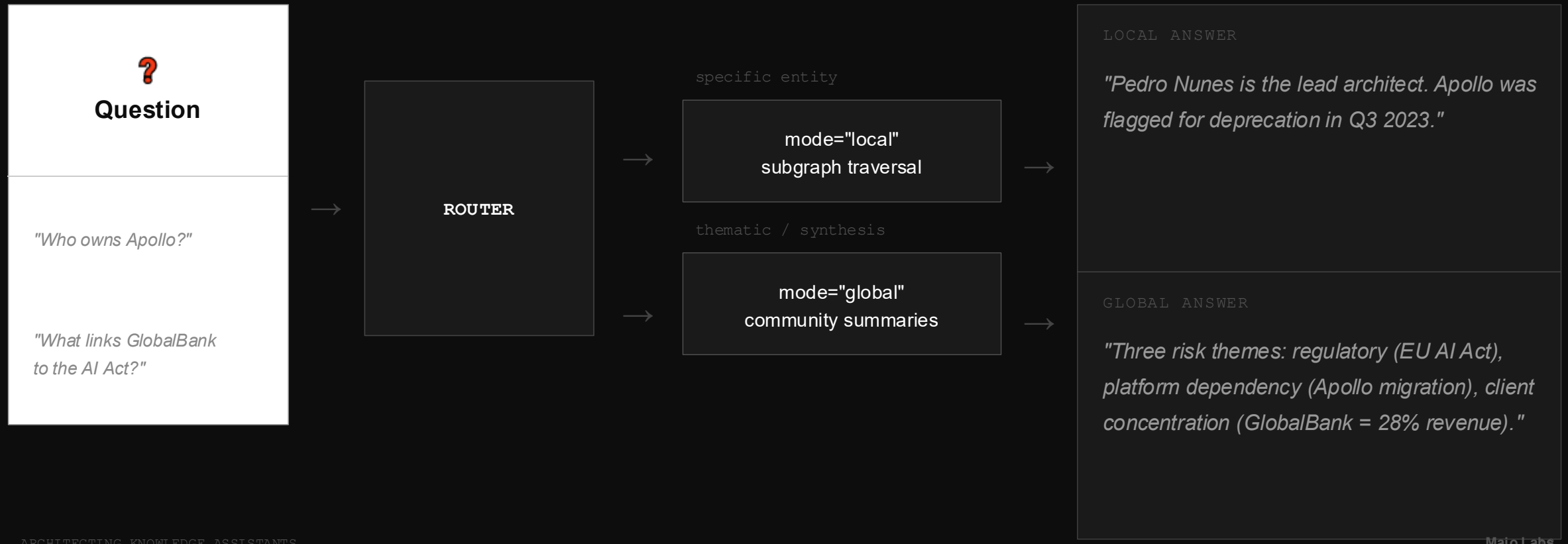
Entity traversal → precise answer with citation.

04 Global query

```
result = await rag.aquery(
  "What are the main risk themes?",
  param=QueryParam(mode="global")
)
```

Community synthesis → answer no chunk can provide.

In the notebook: the question still decides.



Five things nobody warns you about.

1 **Routing wrong is worse than not routing**

Log every decision. A mis-routed question to GraphRAG burns tokens for nothing.

2 **Memory poisoning is real**

Write verbatim. "User prefers bullets" is safe. "User dislikes detail" causes drift.

3 **Rerankers are worth their latency**

80ms of reranking saves 800ms on the wrong context.

4 **Clean data beats clever retrieval**

Strip the noise before you index. For emails, drop headers, signatures and quoted replies — store only the body. Garbage in, garbage retrieved.

5 **Observability beats accuracy**

You can't fix what you can't see. Trace every routing decision. LangSmith or Arize.

What you are about to build.

	RAG 101	WHAT YOU WILL BUILD
RETRIEVAL	One vector store — everything	Router → 4 surfaces → right tool
REASONING	Chunk similarity only	Multi-hop over a knowledge graph
MEMORY	Stateless — resets every query	Session + user + world (writes back)
QUESTIONS	Factual, single-document	Relational, thematic, cross-document

What we use. Why each piece.

COMPONENT	FUNCTION	DETAIL	BOX
LangGraph	Orchestration + state	Router and memory state machine	1+5
LightRAG	GraphRAG engine	pip install lightrag-hku	2
gpt-4o-mini	LLM — routing + generation	Claude Sonnet 4 also supported	1+4
Chroma	Vector store	In-memory, zero infra	2
DuckDB	SQL — structured data	Sales, KPIs, live numbers	2
Tavily	Web search	Real-time, free tier	2
NetworkX	Knowledge graph (dev)	LightRAG default — zero setup	2

No Python required. Neo4j + LLM GraphBuilder.

For teams that want GraphRAG without writing Python. AuraDB Free + LLM Graph Builder — upload PDFs, visualise the graph, query in natural language.

01

Create AuraDB Free account

console.neo4j.io → free tier: 1 graph, 50k nodes.

02

Open LLM Graph Builder

Drag-and-drop PDFs. Choose LLM: GPT / Gemini / Claude.

03

Automatic graph generation

Entities and relations extracted. Interactive visualisation. Zero code.

04

Chat over the graph

Natural language → Cypher → grounded answer with citations.

The graph does not replace RAG.

It is what RAG becomes when the questions get serious.